

C950 Task 1 — Algorithm Overview

WGUPS Routing Program Planning

Student: bwhi880

September 19, 2026

Algorithm Identification

The algorithm selected for the WGUPS Routing Program is the **Clarke-Wright savings algorithm** with **2-opt local search** (“Vehicle routing problem,” n.d.; “2-opt,” n.d.). The Clarke-Wright algorithm is a self-adjusting construction heuristic designed for the Vehicle Routing Problem. It begins with each package as its own round-trip from the hub, precomputes a savings value for every pair of packages — the mileage reduction achieved by visiting both stops in a single route rather than making separate trips — and processes the sorted savings list in descending order, fusing the two routes that yield the greatest mileage reduction whenever the combined route remains within the truck’s capacity. Because the set of viable merges shrinks dynamically after each combination and previously valid merge candidates become invalid once packages join the same route, the algorithm is classified as self-adjusting. After construction, a 2-opt local search polishes each truck route by removing edge crossings: it repeatedly examines pairs of edges in the route and, if reconnecting them the other way shortens the distance, reverses the segment between them (“2-opt,” n.d.). This two-phase approach — global construction followed by local improvement — is appropriate for the WGUPS scenario because it natively handles multiple trucks, respects the 16-package capacity limit, and produces routes short enough to keep total mileage under 140 miles.

Data Structure Identification

A **chaining hash table** is the self-adjusting data structure chosen to store package data. The table maps a package ID to the bucket containing the full package record. Chaining resolves collisions by linking multiple records in the same bucket without probing for open slots. This avoids clustering and guarantees insertion will succeed even when the load factor is high.

Explanation of Data Structure Relationships

The hash table stores each package as a composite record containing the following fields:

- `package_id` — the unique integer identifier and hash key.
- `address, city, zip` — the destination data.
- `deadline` — the promised delivery time (e.g., “9:00 a.m.,” “10:30 a.m.,” or “EOD”).
- `weight` — package weight in kilograms.
- `status` — a state label (delayed, at hub, en route, delivered) paired with a timestamp.
- `special_notes` — constraints such as “truck 2 only” or “wrong address”.

The hash table models the relationship among these components by using `package_id` as the primary key. All other fields are stored as a single value object attached to that key, so an $O(1)$ average lookup by ID automatically retrieves the entire record. Because delivery management requires frequent status updates and deadline lookups, the hash table provides the fastest random access available for this dataset. The self-adjusting nature stems from dynamic resizing: when the load factor exceeds a threshold (0.75 for example), the table allocates a larger array, rehashes all existing keys, and redistributes records, keeping operations efficient as the dataset grows (“Hash table,” n.d.).

Program Overview

C1. Algorithm Logic (Pseudocode)

The pseudocode below describes the route-planning logic used by each truck.

```
PROCEDURE ClarkeWrightRoutes(hub, packages, distance_table, trucks)
  // Phase 1: Each package begins as its own route (hub → pkg → hub)
  routes ← []
  FOR i ← 0 TO LEN(packages) - 1 DO
    new_route ← [packages[i]]
    APPEND(routes, new_route)
  NEXT i

  // Phase 2: Compute savings for every pair of packages
  savings ← []
  FOR i ← 0 TO LEN(packages) - 1 DO
    FOR j ← i + 1 TO LEN(packages) - 1 DO
      a ← packages[i].address
      b ← packages[j].address
      s ← distance_table[hub][a] + distance_table[hub][b] - distance_table[a][b]
      APPEND(savings, (s, packages[i], packages[j]))
    NEXT j
  NEXT i
  SORT(savings, descending by s)

  // Phase 3: Iteratively merge routes with highest valid savings
  FOR each (s, pkg_a, pkg_b) in savings DO
    route_a ← FIND_ROUTE(routes, pkg_a)
    route_b ← FIND_ROUTE(routes, pkg_b)
    IF route_a ≠ route_b AND VALID_MERGE(route_a, route_b, trucks) THEN
      merged ← MERGE_AT_ENDPOINTS(route_a, route_b, pkg_a, pkg_b)
      REMOVE(routes, route_a)
      REMOVE(routes, route_b)
      APPEND(routes, merged)
    ENDF
  NEXT

  RETURN routes
ENDPROCEDURE

// Returns TRUE if combined route respects truck capacity and constraints
FUNCTION VALID_MERGE(route_a, route_b, trucks)
  RETURN LEN(route_a) + LEN(route_b) ≤ trucks.capacity
ENDFUNCTION

// Polishes a single route by removing edge crossings
PROCEDURE TwoOpt(route, distance_table)
  improved ← TRUE
  WHILE improved = TRUE DO
    improved ← FALSE
    FOR i ← 0 TO LEN(route) - 3 DO
```

```

    FOR j ← i + 2 TO LEN(route) - 1 DO
        d_before ← distance_table[route[i]][route[i+1]]
                    + distance_table[route[j]][route[j+1]]
        d_after  ← distance_table[route[i]][route[j]]
                    + distance_table[route[i+1]][route[j+1]]
        IF d_after < d_before THEN
            REVERSE_SEGMENT(route, i + 1, j)
            improved ← TRUE
        ENDIF
    NEXT j
NEXT i
ENDWHILE
RETURN route
ENDPROCEDURE

// Simulates driving each route and updates package status/timestamps
PROCEDURE DeliverPackages(trucks, routes, distance_table)
    total_miles ← 0
    FOR t ← 0 TO LEN(trucks) - 1 DO
        current_time ← trucks[t].departure_time
        current_location ← hub
        FOR i ← 0 TO LEN(routes[t]) - 1 DO
            miles ← distance_table[current_location][routes[t][i].address]
            total_miles ← total_miles + miles
            current_time ← current_time + (miles / trucks[t].speed)
            routes[t][i].status ← "delivered at " + current_time
            current_location ← routes[t][i].address
        NEXT i
        // Return to hub
        total_miles ← total_miles + distance_table[current_location][hub]
    NEXT t
    RETURN total_miles
ENDPROCEDURE

```

Distance table indexing: The distance table is keyed by delivery address, not by package ID, because distances describe relationships between physical locations. A package ID is an arbitrary integer with no spatial meaning; therefore, the pseudocode resolves each package to its address before querying the distance table. The package object itself is still carried through the merge process to preserve all metadata (deadline, weight, special notes) for constraint checking and status updates.

Constraint handling: Before route construction, packages are pre-filtered into eligibility pools. “Truck 2 only” packages are forced into Truck 2’s pool. Delayed packages and package 9 are excluded from the initial loading and are inserted when their constraints are satisfied (delay cleared; address corrected at 10:20 a.m.). Because only two drivers are available, trucks are dispatched in waves: Trucks 1 and 2 depart at 8:00 a.m.; Truck 3 departs when the first truck returns to the hub and a driver becomes available.

C2. Development Environment

The WGUPS Routing Program is implemented in **Python 3.12** and executed on **PyPy** for improved just-in-time compilation performance. The development environment consists of:

- **Software:** The project is developed entirely within a reproducible **Nix** environment using flakes (`nix develop --extra-experimental-features flakes`), ensuring identical dependencies across machines. Source code is edited in **Neovim** (configured with **pyright** LSP, **ruff** linter/formatter, and tree-sitter syntax highlighting). Build and execution tasks are orchestrated via a **justfile** (via the `just` command runner), which encapsulates commands to run the program, execute tests, and export the hash table state. Version control is managed with **Git**. No external routing or optimization libraries are used; all data structures are written from scratch to satisfy the task constraints.
- **Hardware:** The application is developed and executed on an Apple MacBook Air (M1, 2020) with an Apple M1 SoC (4 performance cores + 4 efficiency cores at 3.20 GHz), 8.00 GiB of unified memory, and integrated Apple M1 graphics. The system runs macOS Sonoma 14.4.1 on arm64. The program is single-threaded and requires negligible memory, so any modern ARM64 or x86-64 machine with Nix installed will execute it identically.

C3. Space-Time Complexity

The major segments of the program and their complexities are:

1. **Distance table load** — Reading the CSV distance table into a 2-D array of size $n \times n$ (where n is the number of unique addresses) requires $O(n^2)$ time and $O(n^2)$ space.
2. **Package file load** — Inserting p packages into a chaining hash table requires $O(p)$ average time and $O(p)$ space.
3. **Route construction (Clarke-Wright)** — Computing all pair-wise savings for p packages evaluates $O(p^2)$ pairs, producing a savings list of size $m = O(p^2)$. Sorting this list requires $O(m \log m) = O(p^2 \log(p^2)) = O(p^2 \log p)$ time. Merging routes is $O(p^2)$ in the worst case. The space required is $O(p^2)$ for the savings list plus $O(p)$ for the routes.
4. **Route improvement (2-opt)** — For a truck route with k stops, each iteration examines $O(k^2)$ edge pairs and checks whether a swap reduces distance. In the worst case, the algorithm runs $O(k^3)$ time before converging to a local optimum. The auxiliary space is $O(k)$.
5. **Delivery simulation** — Iterating over all truck routes is $O(p)$ time and $O(1)$ extra space.
6. **Status lookup** — Hash table retrieval by `package_id` is $O(1)$ average time and $O(1)$ space.

Because $p = 40$ packages and $k \leq 16$ per truck, the entire program runs in $O(n^2 + p^2 \log p + k^3)$, which simplifies to $O(n^2)$ since the distance table dominates. The overall space complexity is $O(n^2 + p^2)$. Under the scenario constants ($n \approx 27$ addresses, $p = 40$), the program is expected to execute in well under one second. The memory footprint is bounded as follows:

Data structure	Size estimate
Distance table (27×27 floats @ 8 bytes)	~ 6 KB
40 package records (~ 200 bytes each, including strings)	~ 8 KB
Hash table (array + chain pointers for 40 entries)	~ 4 KB
Savings list ($C(40, 2) = 780$ entries @ ~ 32 bytes)	~ 25 KB
2-opt auxiliary arrays per route (max 16 stops)	~ 1 KB
Total	~ 44 KB

Even with PyPy's JIT compiler overhead and object model, the working set remains well under one megabyte.

C4. Scalability and Adaptability

The solution scales with the number of packages and delivery locations through the following design choices:

- **Hash table:** Average $O(1)$ lookups are invariant to the number of packages stored, provided the table is resized when the load factor exceeds a fixed threshold. Dynamic rehashing guarantees that insertion and lookup remain $O(1)$ average even as the package count grows from 40 to thousands.
- **Distance table:** The table is indexed by address strings (or integer location indices), so adding new locations is a matter of appending rows and columns. The memory footprint grows quadratically with number of locations, but modern systems can easily manage city-scale distances (hundreds of locations) in RAM.
- **Clarke-Wright construction:** The $O(p^2 \log p)$ time is acceptable while p stays in the hundreds. For thousands of packages, the savings list can be pruned to only evaluate the top- m highest-saving pairs, or the algorithm can be replaced with a sweep-based constructor.
- **2-opt improvement:** The $O(k^3)$ polishing cost remains trivial while $k \leq 50$. For larger routes, the algorithm can be upgraded to **Or-opt** (segment relocation) or Lin-Kernighan to escape shallow local optima.
- **Algorithm swap:** Because construction and improvement are isolated in separate modules, replacing Clarke-Wright with another constructor (e.g., nearest neighbor, sweep) or 2-opt with a more powerful polisher requires changing only that routine.

C5. Software Efficiency and Maintainability

The software design is efficient and maintainable for several reasons:

- **Separation of concerns:** Data loading (CSV parsing), data storage (hash table), route construction (Clarke-Wright), route improvement (2-opt), and delivery simulation (time/mileage tracking) are implemented in distinct modules. A developer can modify the hash table probing strategy or the routing heuristic without touching unrelated code.
- **Single responsibility:** Each class or function has one clear purpose. For example, the `HashTable` class only manages key-value storage; the `RoutePlanner` only computes routes; the `Truck` class only tracks state and mileage.
- **Readability:** All identifiers are descriptive (`package_id`, `delivery_deadline`, `distance_to_hub`), and docstrings explain public method contracts. This reduces the onboarding cost for new developers or reviewers.
- **No external dependencies:** By avoiding third-party routing libraries, the project remains portable and avoids dependency-version conflicts.

C6. Strengths and Weaknesses of the Self-Adjusting Data Structure

Strengths:

- **Average-case $O(1)$ operations:** Insertion, deletion, and lookup are constant-time when the hash function distributes keys uniformly.
- **Efficient updates:** Changing a package's status from "en route" to "delivered" is a single hash lookup followed by an in-place mutation.
- **Dynamic resizing:** The table grows automatically as packages are added, so the programmer does not need to predict the maximum dataset size in advance.

Weaknesses:

- **Worst-case $O(n)$:** If every package ID hashes to the same bucket (poor hash function or adversarial input), the chain degrades to a linked list and all operations become linear.
- **Space overhead:** Chaining stores pointers for every collision node, which consumes more memory than open addressing for the same load factor.
- **No ordering:** The hash table does not support range queries or sorted traversal. If the supervisor wants a list of all packages sorted by deadline, the data must be copied into a list, and sorted separately, adding $O(p \log p)$ overhead.

C7. Justification of the Key

For efficient delivery management, the **package ID** is the optimal key. The rationale is:

- **Delivery address:** Not unique — multiple packages may share the same address. Using it as a key would require mapping one address to a list of packages, complicating $O(1)$ status lookups.
- **Delivery deadline:** Not unique and subject to change. A deadline-based key would force the system to restructure indices whenever a deadline is corrected (e.g., the 10:20 a.m. address update for package 9).
- **Delivery city / zip code:** Far too coarse; thousands of packages may share a zip code, making it a poor discriminator.
- **Package weight:** Not unique and irrelevant for lookup operations.
- **Delivery status:** Highly volatile. A status key would be useless because the value changes continually during the day.
- **Package ID:** Guaranteed unique, immutable, and small. It maps directly to a single package record, enabling $O(1)$ lookup by the supervisor at any time. Because the task explicitly requires looking up packages by ID, this choice is both a functional requirement and a performance optimization.

References

Vehicle routing problem. (n.d.). In **Wikipedia**. https://en.wikipedia.org/wiki/Vehicle_routing_problem

2-opt. (n.d.). In **Wikipedia**. <https://en.wikipedia.org/wiki/2-opt>

Hash table. (n.d.). In **Wikipedia**. https://en.wikipedia.org/wiki/Hash_table